SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 07

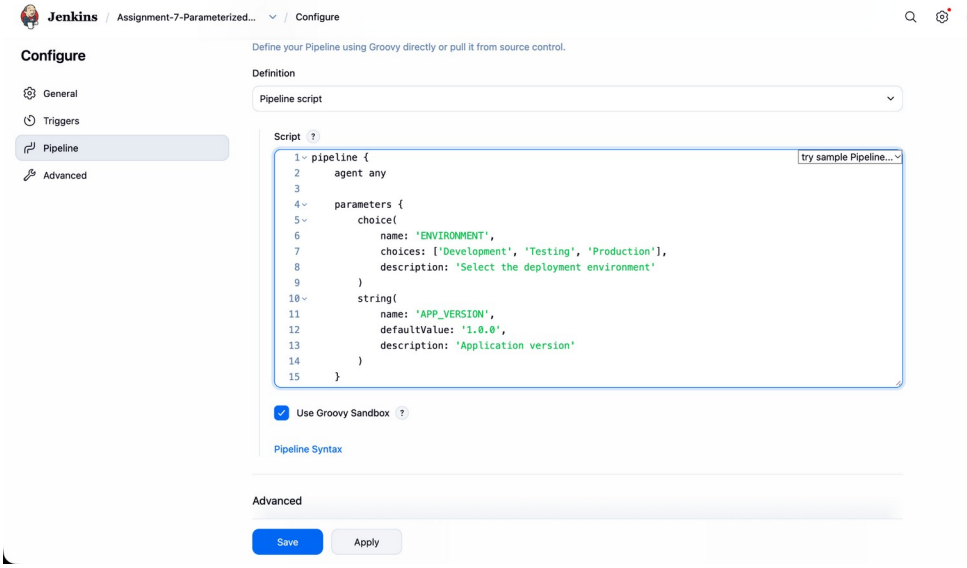
Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Pushkraj P Naik**
PRN No.	**23070122169**
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025 – 2026, Semester VII
Date of Performance	23rd July, 2026
Title of Assignment:	Pipeline with Parameters
Theory:	**Declarative Pipeline with Parameters** A Declarative Pipeline is a Jenkins feature that allows the build, test, and deployment process to be defined as code using a structured Groovy syntax. It provides a simple and readable way to automate software development workflows. Pipeline parameters enable users to provide input values at the time of execution, making the pipeline flexible and reusable. In this experiment, a string parameter (GREETING_NAME) was added to the pipeline, allowing the user to enter a custom value during execution. The pipeline accessed this value using the params object and displayed it in the console output. This demonstrates how parameterized pipelines can accept user input and perform dynamic operations based on the provided values.

Implementation:	**Step 1: Open the Existing Pipeline Project**



Opened Pipeline Configuration — Opened the configuration page of the existing Declarative-Pipeline-Demo project to modify the pipeline script and add build parameters.

Step 2: Add the Parameters Block & Save



Modified Script:

```
pipeline {
    agent any

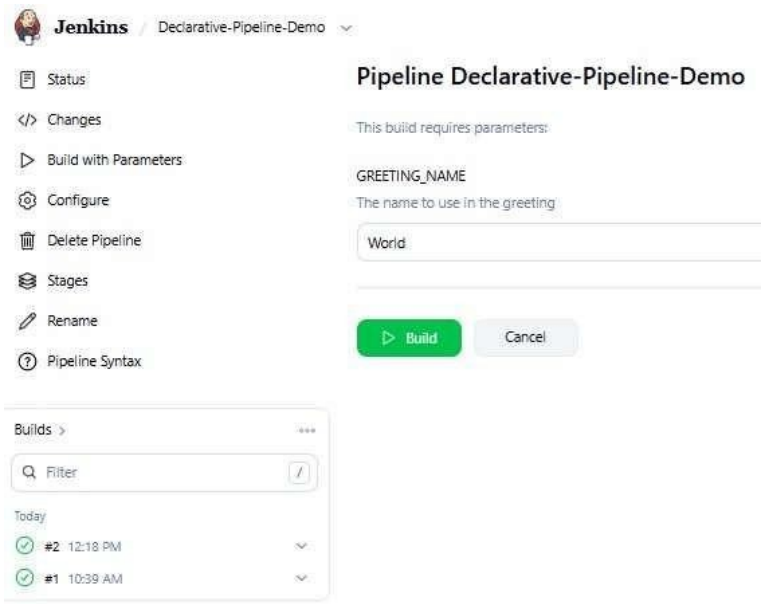
    parameters {
        choice(
            name: 'ENVIRONMENT',
            choices: ['Development', 'Testing', 'Production'],
            description: 'Select the deployment environment'
        )
        string(
            name: 'APP_VERSION',
            defaultValue: '1.0.0',
            description: 'Application version'
        )
    }

    stages {
        stage('Build') {
            steps {
                echo "Building version ${params.APP_VERSION}"
            }
        }

        stage('Deploy') {
            steps {
                echo "Deploying to ${params.ENVIRONMENT}"
                echo "Application version: ${params.APP_VERSION}"
            }
        }
    }
}
```

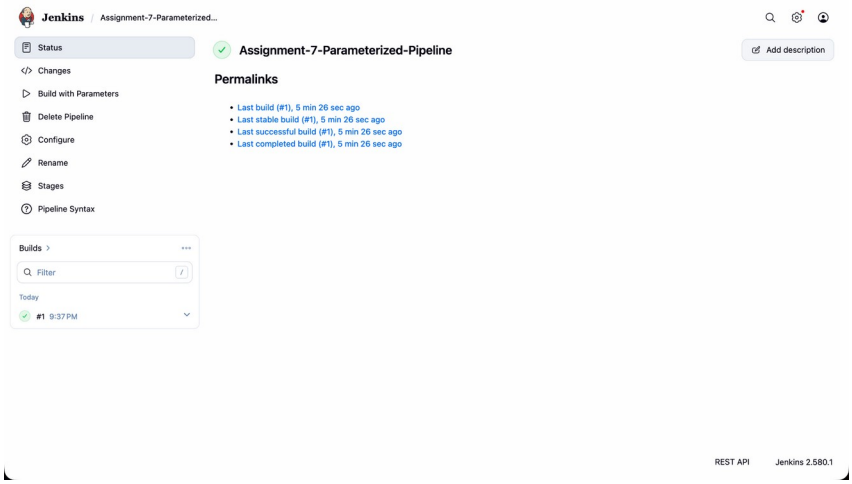
Configured Parameterized Pipeline — Added the GREETING_NAME string parameter to the Declarative Pipeline, updated the pipeline script to use the parameter in the greeting message, and saved the updated pipeline configuration.

Step 3: Build with Parameters



Provided Build Parameter — Used the Build with Parameters option and entered the value for the GREETING_NAME parameter before executing the Declarative Pipeline.

Step 4: Execute the Parameterized Pipeline



Executed Parameterized Pipeline — Successfully executed the parameterized Declarative Pipeline using the provided GREETING_NAME value and verified that the build completed successfully.

Step 5: Verify the Parameter Usage

```
[Pipeline] echo
Git checkout completed.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Run Example Command)
[Pipeline] echo
Hello World
[Pipeline] sh
+ echo Hello from Declarative Pipeline!
Hello from Declarative Pipeline!
[Pipeline] sh
+ echo --- Reading README ---
--- Reading README ---
[Pipeline] sh
+ cat README.md
<<<< My Awesome DevOps Lab (Resolved Conflict)


This update adds details about a new feature.

[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline finished.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Verified Parameter Usage — Examined the console output and verified that the pipeline successfully used the GREETING_NAME parameter value (World) during execution. The pipeline completed successfully with a Finished: SUCCESS status.